

Sydney Residential Price Estimation: A Machine Learning Valuation System Under Market Change

Hangyu Hu

14735221

Notebook:

https://colab.research.google.com/github/AlexHu1210/uts-ml-a2-sydney-house-price/blob/main/notebook/UTS_ML_A2_Sydney_House_Price.ipynb

GitHub:

<https://github.com/AlexHu1210/uts-ml-a2-sydney-house-price>

1 Problem Definition

1.1 Background and Motivation

Estimating how much a residential property can sell for is relevant to mortgage approval, pricing decisions by buyers and sellers, and property websites. A common industry approach is an Automated Valuation Model (AVM), which uses property attributes and location to estimate a transaction price statistically. In Australia, Domain, CoreLogic, and PropTrack publish AVM estimates, and banks also use AVMs in place of some physical valuations. Zillow reports the proportion of Zestimate predictions that fall within $\pm 10\%$ of the actual sale price (Zillow, 2011). I use a similar type of metric later in this report.

Simple fixed rules are not sufficient for this task. The same four-bedroom house might sell for around AUD 500,000 on the Central Coast but for more than AUD 10 million near the harbour, and the overall Sydney market rose by roughly one third from early 2020 to late 2021. A suburb-median rule ignores the property itself, while a price-per-square-metre rule ignores location; neither adjusts to changes in the cash rate. A human valuer also cannot revalue ten thousand properties every quarter. This dataset contains 10,864 labelled transactions with features like those a valuer would consider, which makes the task suitable for supervised learning.

1.2 Task Description

The task is to estimate the sale price of a residential property in Greater Sydney when the user provides features that would be known at, or before, the time of sale. This is a supervised regression problem because each training record has a numerical target, price. The models are trained on $\log(\text{price})$; the reason is explained in Section 1.5 and tested experimentally in Section 8 of the notebook.

1.3 Inputs and Outputs for Training and Deployment

	Training phase
Input	One transaction record: property attributes (num_bed, num_bath, num_parking, property_size, type), suburb-level attributes (suburb, suburb_population, suburb_median_income, suburb_sqkm, suburb_lat, suburb_lng, suburb_elevation, km_from_cbd), sale date (date_sold, split into year and month), and two macroeconomic series (cash_rate, property_inflation_index).
Target	price: positive integer, transaction price in AUD. The model fits $\log(\text{price})$.
Example	A 4-bedroom, 2-bathroom, 594 m ² House in Halekulani, 78.5 km from the CBD, sold on 2016-01-13 $\rightarrow$ target 525,000.

	Deployment phase
Input	The same type of attributes for a property that has not yet sold. property_inflation_index must be the latest value already published, not a future value (a potential source of leakage tested in notebook Section 8, experiment C').
Output	A point estimate of sale price in AUD (obtained by exponentiating the predicted log value), a q10–q90 interval from the quantile models, and a relatively conservative lending reference value (q25).

	The estimate_price() function in notebook Section 11 implements this interface.
--	---

1.4 Cleaned Input Specification

The table below is generated directly from the cleaned data frame and shows the actual ranges seen by the models. The units and notes describe the information a third party would need to construct valid inputs.

Field	Type	Unit	Cleaned range	Notes
num_bed	int	count	0–10	Number of bedrooms; studio properties may have 0
num_bath	int	count	0–7	Number of bathrooms
num_parking	int	count	0–8	Number of parking spaces
property_size	int	m ²	29–10,000	Land area for Houses; internal/building area for units
type	categorical	—	10 categories	Residential property type; 86% are Houses
suburb	categorical	—	636 categories	A typical suburb has about 16 transactions
suburb_population	int	people	22–47,176	Census data at suburb level (census year not provided)
suburb_median_income	int	AUD/year	14,248–97,500	Census data at suburb level
suburb_sqkm	float	km ²	0.089–87.154	Suburb area
suburb_lat / suburb_lng	float	degrees	–34.11 to –33.16 / 150.55 to 151.57	Suburb centroid, not the exact property location
suburb_elevation	int	m	0–405	Suburb-level value
km_from_cbd	float	km	0.31–84.79	Distance from suburb centre to Sydney CBD
date_sold	date	—	2016-01-13 to 2022-01-01	Split into year and month
cash_rate	float	% p.a.	0.1–2.0	RBA cash rate on the sale date
property_inflation_index	float	index	150.9–220.1	Quarterly property price index; may be published with a lag
price	int	AUD	272,500–17,000,000	Target; model fits log(price)

1.5 Data Source and Cleaning

The dataset file domain_properties.csv comes from the Sydney House Prices dataset published by Kaggle user alexlau203 (alexlau203, n.d.). It contains 11,160 Greater Sydney property transactions from January 2016 to January 2022, scraped from Domain.com.au, together with suburb-level census

information and two macroeconomic series. The dataset has 17 columns and no missing values, so no imputation is required. The main data-quality issue is implausible values rather than missing data.

Initial exploration identified several characteristics that directly affected the model design. First, the price distribution is strongly right skewed. In the raw data, the median price is AUD 1,388,000, with values ranging from AUD 225,000 to AUD 60,000,000, so a small number of expensive properties create a long right tail. After a log transformation, the distribution becomes much closer to symmetric. For this reason, the later models use $\log(\text{price})$ as the prediction target and are evaluated mainly with relative-error metrics. Second, the location information only represents the suburb level. The latitude and longitude are suburb centroids, so all properties in the same suburb share the same coordinates. Across 637 suburbs, a typical suburb contains only about 16 transactions, which is not enough to support street-level price modelling. Finally, the Sydney housing market changed substantially during the period covered by the data. The cash rate fell from 2.0% in 2016 to 0.1% in 2020–2021, while the property price index rose from about 166 in early 2020 to 220.1 by the end of 2021. The year 2021 alone contains 5,328 records, close to half of the dataset, and corresponds to a strong market upswing. Therefore, the experiments need to account for changes across years rather than if all transactions come from the same market environment.

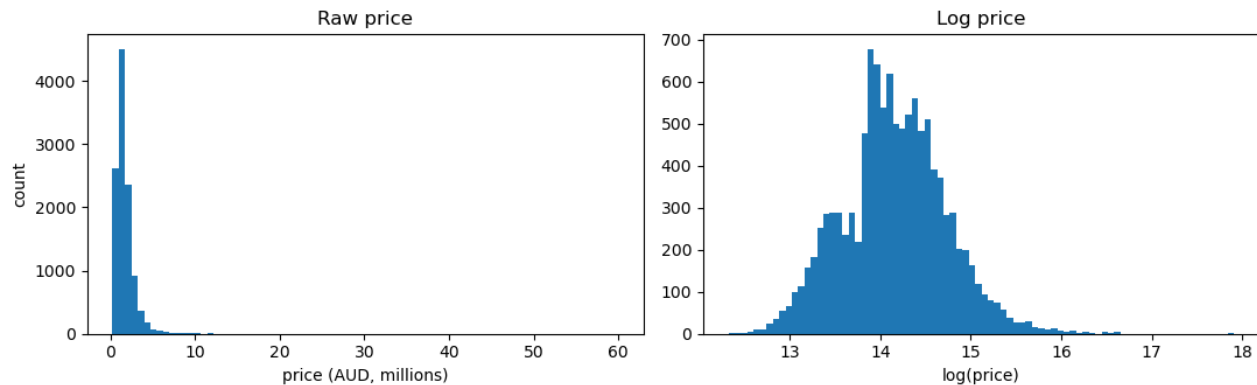


Figure 1. Distribution of raw price (left) and log price (right). The log transformation makes the distribution much more symmetric, which motivates training on $\log(\text{price})$.

The cleaning rules are based on the prediction scope of this project rather than simply removing all statistical outliers. Because the task is to value a single residential property, I first removed property types outside this scope: Vacant Land, New Land, Development Site, Rural, Acreage / Semi-Rural, and Block of Units, for a total of 235 records. I then checked the numbers of bedrooms, bathrooms, and parking spaces, setting upper limits of 10, 8, and 8 respectively. The data contain several clearly implausible records, such as a property labelled House with 47 bedrooms and 46 bathrooms, so 51 records were removed at this stage.

Property size was restricted to 20–10,000 m², removing 10 clearly abnormal records such as a House with only 7 m² and an Apartment with 20,695 m². I did not impose an upper limit on sale price because a very high price does not necessarily indicate bad data. For example, a 3-bedroom, 325 m² property in Woollahra sold for AUD 17 million, and such a transaction may still be genuine. I therefore kept these high-value properties and used $\log(\text{price})$ as the target to reduce their influence during model training. After cleaning, 10,864 records remained, representing 97.3% of the original dataset.

1.6 Research Question

The test data come from the second half of 2021, when the overall market price level was already higher than in the 2016–2020 training period. The central research question is therefore: can a model trained on historical transactions from 2016–2020 accurately predict house prices in the 2021 market?

I then test two ways of helping the model adapt to the 2021 price increase. The first is to provide the cash rate and property price index directly as input features so that the model learns the relationship between market conditions and price. The second is to use the property price index to adjust the prediction target so that the model mainly learns the property's value relative to the overall market level. To better reflect real deployment, the data are split chronologically rather than randomly, allowing the model to be evaluated on its ability to use past data to predict a later market.

2 Machine Learning Approach

2.1 System Overview

The complete workflow from raw data to the final price prediction is as follows:

The 10,864 cleaned records are split by transaction date into training, validation, and test sets. To make the comparison fair, all candidate models use the same core input information and differ mainly in the model itself. Model and hyperparameter choices are made using the validation set, while the test set is used only for final evaluation so that test data do not influence model design or selection.

The final model does not learn the sale price directly. Instead, it uses the property price index to separate overall market movements from property-specific value. This allows the model to focus mainly on relationships between property attributes, location, and relative value. The adjusted target is $\log(\text{price}) - \log(\text{property_inflation_index})$. At prediction time, the current market index is added back to recover the final price estimate in Australian dollars.

In addition to a point prediction, I train Quantile LightGBM models for several price quantiles to obtain a prediction interval. The complete prediction pipeline is wrapped in an `estimate_price()` function. Given basic property information and the current property price index, the function returns a point estimate, an 80% prediction interval, and a more conservative q25 reference value.

Step	Content	Notebook section
Data loading	Read Sydney property transaction data, 11,160 records in total	§1
Cleaning	Remove non-residential property types and clearly implausible records, leaving 10,864 records	§2
Data split	Chronological split: 2016–2020 training, first half of 2021 validation, second half of 2021 test	§3
Evaluation	Compare predicted and true prices using PPE10 (share within 10% error) and MdAPE (median relative error), with simple baselines for reference	§4
Model training	Train and compare Ridge, LightGBM, and a neural network	§5–7

Final-model improvement	Use the property price index to reduce the effect of overall market movements	\$8
Further testing	Compare different loss functions and add prediction intervals	\$9
Error analysis	Analyse where the model performs well or poorly by price, distance, and property type	\$10
Deployment	Wrap the final model in <code>estimate_price()</code> ; property information can be entered to return a price estimate and interval	\$11

2.2 Chronological Data Split

In a real application, a house-price model can only use transactions that have already occurred to predict later prices. For this reason, the data are split by sale date rather than randomly. The training set contains 5,629 transactions before 2021 with a median price of AUD 1.11 million. The validation set contains 1,328 transactions from January to June 2021 with a median of AUD 1.73 million. The test set contains 3,907 transactions from July 2021 onward with a median of AUD 1.65 million. Because Sydney prices rose sharply in 2021, I use the first half of the year for validation and the second half for testing, so model selection can observe some post-increase data while the final test still represents a later period.

For comparison, I also run a random-split experiment, but only as a leakage control. In a random training set of the same size, 47% of the records come from 2021 or later, and the median training price rises from AUD 1.11 million under the chronological split to AUD 1.38 million. This makes the random training data more similar to the test period because some future-market records are mixed into training. The random split is therefore used only to show the effect of the split strategy and is not used to train or select the final model.

2.3 Evaluation Metrics

For house-price prediction, relative error is more informative than absolute dollar error alone. For example, an AUD 100,000 error on a AUD 1 million property is 10%, whereas the same AUD 100,000 error on a AUD 10 million property is only 1%. Therefore, the main evaluation metrics are PPE10 and MdAPE, with median bias and RMSE used as supporting metrics.

For each property, APE (Absolute Percentage Error) is the percentage difference between the predicted price and the true sale price:

$$\text{APE} = |\text{predicted price} - \text{true price}| / \text{true price}$$

Metric	Definition	Purpose
PPE10	Proportion of properties with prediction error within $\pm 10\%$	Primary metric. Higher values mean more valuations are close to the true sale price
MdAPE	Median percentage error across all properties	Primary metric. Represents the prediction error for a “typical” property; lower is better
Median bias	Overall tendency for predicted prices to be above or below true prices	Shows systematic overestimation or underestimation; negative means underestimation and positive means overestimation

RMSE (log)	Error between predicted and true prices on the log scale	Measures error on the model training target; lower is better
RMSE (AUD)	Dollar error between predicted and true sale prices	Reported as a supporting metric, but strongly affected by a small number of expensive properties

It is important to note that the loss optimised during training is not exactly the same as the practical evaluation objective. A lower training loss does not necessarily mean that more properties are predicted within $\pm 10\%$ of the true price. This issue is examined further in Section 2.9.

2.4 Baselines: What Can Be Done Without a Machine Learning Model?

Looking only at the machine learning models makes it difficult to judge whether their performance is actually useful. For example, $\text{PPE}_{10} = 40\%$ has little meaning without a simple reference. I therefore create three training-set baselines: the overall median sale price, the median price within each suburb, and the median price within each suburb–property-type combination. For suburbs not seen in training, the overall median is used as the prediction.

These baselines do not use machine learning. They provide a simple benchmark: if the machine learning models cannot clearly outperform them, then the additional model complexity is not providing enough value.

2.5 Model 1: Ridge Regression

Ridge regression is the first machine learning model in this project and serves as an important comparison for the more complex models. All three machine learning models use the same data and broadly similar preprocessing. Numerical features include property attributes, suburb statistics, `km_from_cbd`, `year`, `month`, `cash_rate`, and `property_inflation_index`. Because `property_size` has a wide range, it is first transformed using `log1p` and then standardised together with the other numerical features. The categorical variables `type` and `suburb` are converted using one-hot encoding. After preprocessing, the model has 616 input features.

Ridge regression can be viewed as linear regression with L2 regularisation (Hoerl & Kennard, 1970). The model predicts $\log(\text{price})$ as a weighted combination of the input features, while the L2 penalty discourages excessively large weights. This is useful here because one-hot encoding creates more than 600 features and some of them are strongly correlated. Regularisation reduces over-reliance on individual features and makes the solution more stable.

To understand the Ridge training process rather than relying only on an off-the-shelf library, I first computed the Ridge closed-form solution directly in NumPy and compared it with the scikit-learn Ridge implementation (Pedregosa et al., 2011). The maximum difference between the two sets of weights was approximately 7.3×10^{-15} , indicating that my implementation matches the library result to numerical precision.

I then manually implemented Gradient Descent (GD) and Adam to optimise the same Ridge loss. Adam is an adaptive optimisation method based on estimates of the first and second moments of the gradient (Kingma & Ba, 2015). In this experiment, Adam was already very close to the closed-form

solution after about 1,500 iterations, whereas standard Gradient Descent still showed a visible gap after 2,000 iterations. Figure 2 shows the convergence of the two methods.

One reason for this difference is that the optimisation difficulty varies substantially across feature directions. The condition number of the Ridge problem in this project is approximately 15,247, indicating that the loss changes rapidly in some directions and slowly in others. Standard Gradient Descent uses the same learning rate for all parameters, so the global step size must remain small enough to be stable. Adam adapts the update magnitude using each parameter's gradient history, which is why it converges faster in this experiment.

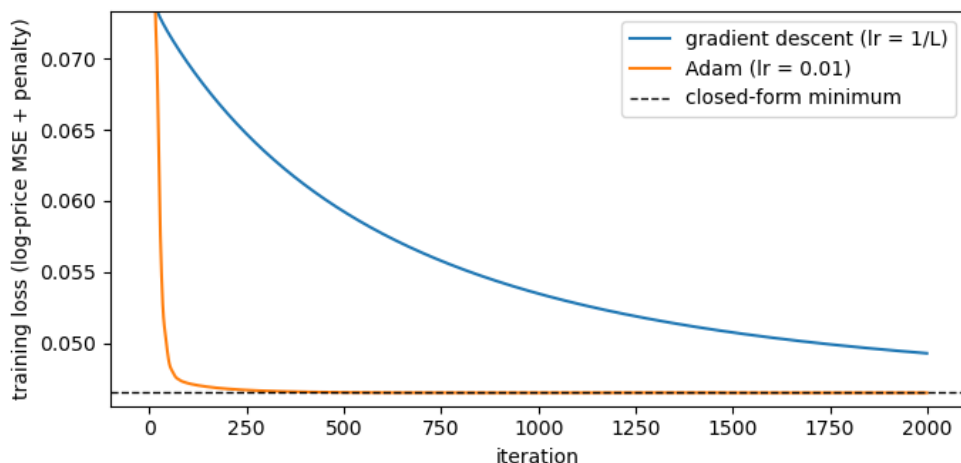


Figure 2. Training process of Gradient Descent and Adam on the same Ridge loss. Adam approaches the closed-form solution faster, while standard Gradient Descent converges more slowly.

Finally, I compare different regularisation strengths α on the validation set. The search range is 0.01–100, and validation PPE10 ranges from 31.7% to 34.8%. The best result is obtained with $\alpha = 1$, so the later Ridge experiments use $\alpha = 1$.

2.6 Model 2: Gradient Boosting Trees (LightGBM)

Ridge regression mainly learns linear relationships between each feature and price, but real house-price relationships are more complex. For example, the effect of adding one bedroom may differ across suburbs, property sizes, and property types. I therefore use LightGBM as the second model so that the system can learn nonlinear relationships and interactions between features automatically.

The basic idea of Gradient Boosting is to add weak learners sequentially so that later learners correct errors left by earlier ones (Friedman, 2001). This project uses LightGBM as the gradient-boosted tree model (Ke et al., 2017). The prediction target remains log(price), and the model is trained with squared error.

To control model complexity, I compare num_leaves values of 7, 15, 31, and 63 on the validation set. The best validation PPE10 is obtained with 15 leaves, so I select num_leaves = 15. I also use early stopping: training stops when the validation error has not improved for 100 rounds, which occurs at round 673 for this model.

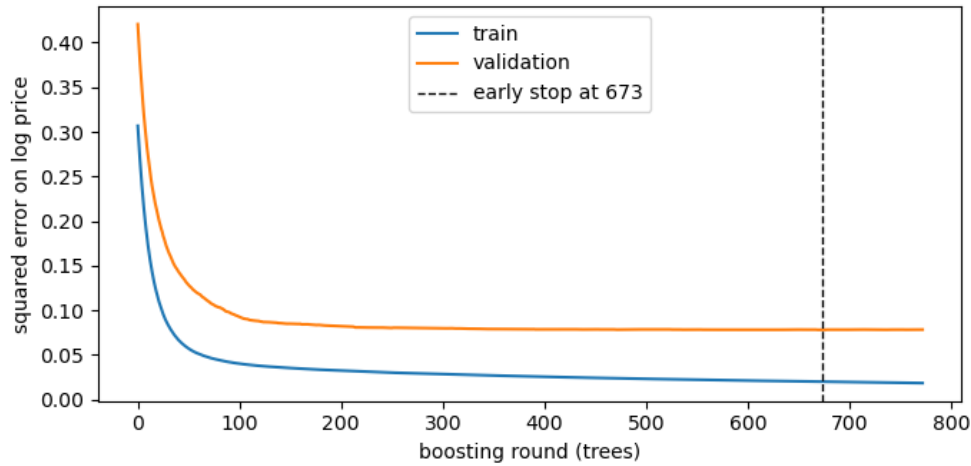


Figure 3. LightGBM training and validation error as the number of trees increases. Training error keeps decreasing while validation error stabilises, so training stops at round 673.

However, LightGBM does not perform as expected on the test set. Test PPE10 is 22.5% with a median bias of -18.2% , compared with Ridge at 33.6% PPE10 and $+8.7\%$ median bias. This indicates that LightGBM substantially underestimates prices in the second half of 2021.

To investigate the reason, I run two additional experiments. First, I replace year, cash_rate, and property_inflation_index in the test data with their end-of-2020 values and predict again. The predictions change almost not at all. This shows that, although these variables are included in the model, LightGBM is not using them to extrapolate the market increase beyond the range seen during training.

Second, I train both Ridge and LightGBM only on data before July 2020 and predict the second half of 2020. In this within-range setting, LightGBM achieves 43.7% PPE10 compared with 37.1% for Ridge. This shows that LightGBM is not always worse than Ridge; the main problem appears when it is asked to predict the already higher 2021 market.

This result shows that the main difficulty in this project is not simply choosing the best model, but handling the change in market level between the training and test periods. This observation motivates the later redesign of the prediction target.

2.7 Model 3: Multilayer Perceptron (PyTorch)

The third model is a multilayer perceptron (MLP) implemented in PyTorch (Paszke et al., 2019). Ridge has a relatively simple structure, while LightGBM can learn more complex nonlinear relationships. To compare a different model family under the same data and features, I add a neural network and evaluate its behaviour.

The MLP uses the same 616 input features as Ridge, with a $616 \rightarrow 128 \rightarrow 64 \rightarrow 1$ architecture and 87,297 trainable parameters. The two hidden layers use ReLU activations, and the final layer outputs predicted log(price). The model uses squared error as the loss and is trained with Adam. To reduce overfitting, I include L2 regularisation and Dropout, together with early stopping: training stops if the validation error has not improved for 30 epochs.

Because neural network training depends on random initialisation and stochastic optimisation, the same model can produce slightly different results across runs. I therefore repeat training with five random seeds. Mean validation PPE10 is $35.3\% \pm 0.9\%$, and mean test PPE10 is $33.7\% \pm 2.8\%$, showing some run-to-run variation. The final MLP uses seed 2 because it performs best on the validation set; the test set is not used for model selection.

The training curves show that training error keeps falling, while validation error starts to increase after about epoch 6, indicating overfitting. Early stopping therefore keeps the model with the best validation performance rather than continuing training to the final epoch. Overall, the MLP test result is close to Ridge, but the training process is more complex and more sensitive to random initialisation.

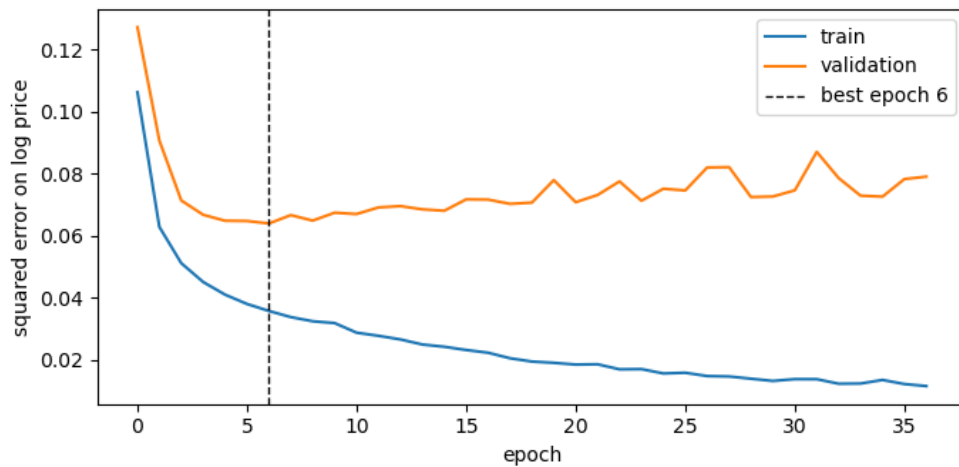


Figure 4. Training and validation error for MLP (seed 2). Training error continues to fall, while validation error starts to rise after epoch 6, indicating that further training would overfit.

2.8 Ablation Experiments and Final Model

The previous experiments show that simply changing the model cannot fully solve the prediction error caused by the 2021 market shift. I therefore run a set of ablation experiments in which one important design choice is changed at a time to identify which factors actually affect performance. LightGBM is used for most ablations, with Ridge added where useful for comparison. All experiments use the same training, validation, and test sets so the results can be compared directly.

Experiment	Change	Question answered
A Prediction target	Compare predicting raw price with predicting $\log(\text{price})$	Does the log transformation improve price prediction?
B Market information	Remove the cash rate, property price index, and time features separately	How dependent is the model on information about market change?
C Index-adjusted target	Use the property price index to remove the overall market level first, then predict relative property value	Is prediction more accurate when market movement is separated from property-specific value?
C' Lagged index	Use the property price index that would have been available one quarter earlier	How much does real-world index publication delay affect the model?

D Suburb	Remove suburb name and keep only distance, income, population, and other regional information	Is the model overly dependent on the specific suburb name?
E Data split	Compare chronological and random splits	Does random splitting look better because it allows the model to see future-period data?

Experiment C is the most important. The earlier results suggest that the overall 2021 market increase is a major source of prediction error. I therefore use the property price index to separate the overall market level from the value associated with the property's own attributes and location.

Assume that the price of a property at time t can be written as:

$$y(t) = I(t) \cdot g(x)$$

where $y(t)$ is the actual sale price, $I(t)$ is the overall market property price index at time t , and $g(x)$ is the relative value determined by property attributes and location. Taking logs gives:

$$\log y(t) - \log I(t) = \log g(x)$$

I therefore change the original target from $\log(\text{price})$ to $\log(\text{price}) - \log(\text{property_inflation_index})$. In this form, the model mainly learns the property's value relative to the overall market, while market-wide increases or decreases are supplied by the index. At prediction time, the current index is combined with the model output to recover the final price estimate.

Based on validation PPE10, the final model is an index-adjusted LightGBM. It no longer uses time features or the specific suburb name. Instead, it uses property attributes, numerical suburb-level information, and property type. After retraining, early stopping selects round 620.

Because the most recent property price index may not yet be available at the time of real deployment, I also keep experiment C'. This version uses an index value that would have been available one quarter earlier, allowing me to simulate a more realistic information delay and measure its effect on performance.

2.9 Training Loss and Practical Objective; Quantile Regression

The practical goal of this project is for predicted prices to be close to actual transaction prices, so PPE10 and MdAPE are the main evaluation metrics. PPE10 is the proportion of properties whose prediction error is within $\pm 10\%$ of the true sale price. However, PPE10 is not suitable as a direct training loss because it only checks whether a prediction falls inside the 10% band and does not change smoothly with error magnitude. For example, errors of 1% and 9% are both counted as correct, while errors of 11% and 60% are both counted as incorrect.

For this reason, model training mainly uses squared error on log prices. Unlike PPE10, squared error changes continuously as the prediction error changes, which makes it suitable for optimisation. Using $\log(\text{price})$ also makes the loss more closely related to relative price error, which is consistent with the percentage-based evaluation objective of this project.

A lower training loss does not necessarily imply better practical metrics. To examine this difference, I analyse the final model predictions on the test set. The largest 5% of squared-log errors account for 41.8% of the total loss, and none of those records fall inside the PPE10 $\pm 10\%$ band. This shows that a small number of very large errors can dominate squared error.

To compare the effect of different training losses, I test L2, L1, Huber, and Fair losses using the same LightGBM setup. Their test PPE10 values range from 37.3% to 38.2%. Fair loss produces the highest PPE10, but it does not produce the lowest RMSE(log). This shows that different losses emphasise different types of error, and improvement in training loss does not translate directly into improvement in PPE10.

Therefore, I use optimisation-friendly losses during training, but compare and select models mainly using PPE10 and MdAPE rather than training loss alone.

2.9.1 Quantile Regression and Prediction Intervals

The previous models return only one predicted price, but real house-price prediction contains uncertainty. Even two properties with similar attributes and locations may sell for different prices because of factors that are not recorded in the dataset. A single point estimate therefore cannot fully represent prediction uncertainty. To address this, I use Quantile Regression. Quantile regression can learn different conditional quantiles by minimising the Pinball Loss (Koenker & Bassett, 1978).

Standard regression mainly gives a central prediction, while quantile regression estimates different positions in the price distribution. I train five quantile models with $\tau = 0.1, 0.25, 0.5, 0.75, \text{ and } 0.9$, corresponding to q10, q25, q50, q75, and q90. Here q50 represents the middle of the predicted distribution, q10 and q90 define a wider prediction interval, and q25 provides a relatively conservative reference value.

Quantile regression is trained using pinball loss:

$$L_{\tau}(y, \hat{y}) = \begin{cases} \tau(y - \hat{y}), & y \geq \hat{y} \\ (1 - \tau)(\hat{y} - y), & y < \hat{y} \end{cases}$$

where y is the true sale price, $\hat{y}$ is the model prediction, and τ is the target quantile. Unlike ordinary squared error, pinball loss penalises over-prediction and under-prediction differently, with the asymmetry determined by τ . By changing τ , the model can learn different positions in the price distribution rather than returning only one central prediction.

Finally, I use q10 and q90 to form the prediction interval and retain q25 as a relatively conservative reference price. In this way, the valuation system can report not only an approximate property value but also a plausible price range, giving a more complete representation of prediction uncertainty.

2.10 Deployment Function

To make the final model usable, I combine the preprocessing and prediction steps in an `estimate_price()` function. The user only needs to provide basic property information, including the numbers of bedrooms, bathrooms, and parking spaces, property size, property type, suburb, and the current property price index. The function then performs the remaining feature processing and calls the final model.

Before prediction, the function checks whether the input is valid. For example, if the bedroom, bathroom, parking, or area values fall outside the ranges used during training, the system reports an input

error. For suburb-related information, the function automatically fills in population, income, distance to the CBD, latitude, and longitude from the stored suburb data, so the user does not need to enter these fields manually.

The function returns three main outputs: a point price estimate, a q10–q90 prediction interval, and a relatively conservative q25 reference price. This allows the user to see both a specific estimate and a reasonable range according to the model. I test the complete pipeline with different property types, suburbs, and property sizes, and the system successfully completes the full input-to-output prediction process.

Building on `estimate_price()`, I also create a simple interactive valuation tool to show how the estimate for the same property changes under different market levels. The user can choose the suburb, property type, numbers of bedrooms, bathrooms, and parking spaces, and property size, and the interface calls the same prediction function to generate results.

For 2016–2021, the tool uses the property price index values available in the dataset. For years after 2022, the dataset does not contain the true index, so I generate a hypothetical market index using an adjustable annual growth rate. Therefore, results after 2022 are only scenario demonstrations showing how the model changes under different assumed market growth rates; they are not genuine forecasts of future house prices.

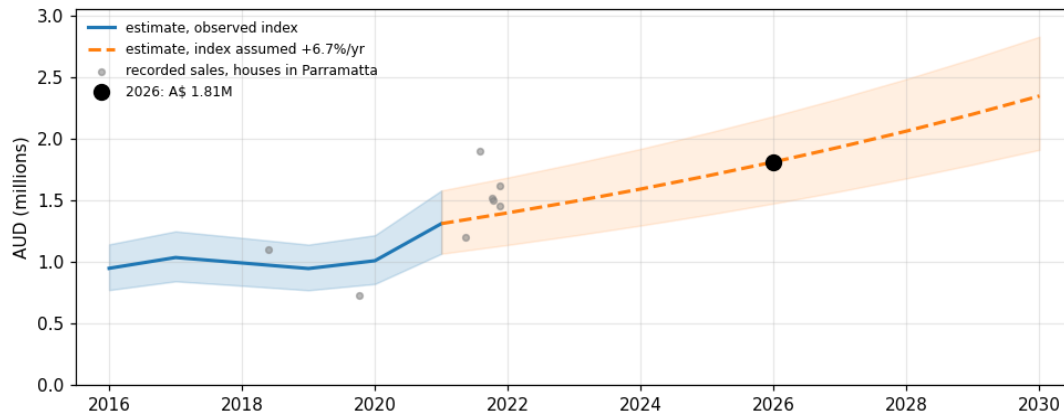


Figure 5. Interactive valuation example. Estimated value of a 3-bedroom property in Parramatta under different market-index levels. The 2016–2021 values use observed index data, while 2022–2030 are scenario results under an assumed growth rate and should not be interpreted as real house-price forecasts.

3 Results

3.1 Main Results

The test set contains 3,907 transactions, mainly from July 2021 onward. All models are trained and selected using only data from before the test period, so the test set evaluates final performance in a later market. Table 1 summarises the test results for the baseline, the three machine learning models, and the index-adjusted models.

Table 1. Test-set results

Model	PPE10	MdAPE	Median bias
-------	-------	-------	-------------

	(%)	(%)	(%)
Baseline: global median	21.1	32.8	−32.5
Ridge	33.6	15.9	+8.7
LightGBM	22.5	20.1	−18.2
MLP	33.6	15.7	+7.6
LightGBM (index-adjusted, final model)	37.9	13.4	−4.7
LightGBM (index-adjusted, one-quarter lag)	35.7	14.5	−7.9

Table 1 shows that the simple global-median baseline performs poorly, with PPE10 = 21.1%, MdAPE = 32.8%, and median bias = −32.5%. This means that using only the overall median of the 2016–2020 training data to predict 2021 prices leads to substantial underestimation, reflecting the higher market level in 2021.

The three machine learning models perform differently. Ridge reaches 33.6% PPE10 and 15.9% MdAPE, clearly improving on the baseline, but its median bias is +8.7%, indicating overall overestimation. LightGBM, when directly predicting log(price), reaches only 22.5% PPE10 and has a median bias of −18.2%, showing substantial underestimation in the 2021 test period. MLP reaches 33.6% PPE10 and 15.7% MdAPE, with overall performance close to Ridge.

After adjusting the target using the property price index, LightGBM improves substantially. The final model reaches 37.9% PPE10, 13.4% MdAPE, and a median bias of −4.7%, compared with −18.2% for the original LightGBM. This indicates that explicitly separating overall market movements is more effective than simply changing the model when dealing with the 2021 market increase.

I also test a one-quarter lag in the property price index to simulate a realistic setting where the latest index has not yet been published. PPE10 falls to 35.7%, MdAPE rises to 14.5%, and median bias becomes −7.9%. Although this is worse than using the contemporaneous index, it is still clearly better than the original LightGBM, showing that the index-adjustment idea remains useful even with publication delay.

3.2 Ablation Results

To understand which design choices affect prediction performance, I vary the prediction target, market information, suburb feature, and data-splitting strategy, then compare the results. Table 2 summarises the main ablation experiments.

Table 2. Ablation experiments (PPE10, MdAPE, and bias are all in %)

Experiment	Model	Validation PPE10	Test PPE10	Test MdAPE	Test bias
Original setting	LightGBM	33.4	22.5	20.1	−18.2
Raw-price target	LightGBM	30.1	24.0	19.9	−15.9
Remove macro variables	LightGBM	24.5	19.4	22.3	−21.2

Remove time and macro variables	LightGBM	22.9	12.2	27.3	−26.7
Index-adjusted target	LightGBM	37.3	37.7	13.5	−4.6
One-quarter lagged index	LightGBM	32.3	36.3	14.5	−6.9
Index-adjusted + remove suburb	LightGBM	39.0	37.9	13.4	−4.7
Original setting	Ridge	34.8	33.6	15.9	+8.7
Index-adjusted target	Ridge	33.4	37.3	14.1	−4.0
Random split	LightGBM	—	42.3	12.1	+1.6

The ablation results show that both the prediction target and market information strongly affect performance. Directly predicting raw price does not improve the result, so later models continue to use $\log(\text{price})$. Removing the macro variables reduces LightGBM test PPE10 from 22.5% to 19.4%. Removing both time and macro variables lowers it further to 12.2%, while median bias worsens to −26.7%. This shows that property and area features alone are not enough to adapt to the overall 2021 market shift.

The largest improvement comes from the index-adjusted target. With this target, LightGBM test PPE10 rises from 22.5% to 37.7%, MdAPE falls from 20.1% to 13.5%, and median bias improves from −18.2% to −4.6%. Ridge also improves from 33.6% to 37.3% PPE10 under the same adjustment. The similar results from two different model families indicate that the main gain comes from target design rather than from the model choice alone.

To account for the fact that the newest index may not yet be available in practice, I also test a one-quarter lag. LightGBM PPE10 is 36.3%, below 37.7% with the contemporaneous index but still clearly above the original model. This shows that index publication delay reduces performance, but the adjustment method remains useful.

Removing the specific suburb name slightly increases LightGBM PPE10 to 37.9%, so the final model does not use suburb as a categorical identifier. Instead, location is represented by latitude, longitude, distance to the CBD, income, and population. Finally, the random split reaches 42.3% PPE10, which is higher than the chronological split. However, the random split mixes future-period transactions into training and makes the train and test distributions more similar, so this result is used only as a control; the project continues to use chronological splitting to evaluate future-market prediction.

3.3 Statistical Reliability of the Results

A single PPE10 or MdAPE value cannot fully show how much the test result might vary, so I use Bootstrap resampling to assess stability. Specifically, I draw 2,000 bootstrap samples with replacement from the 3,907 test transactions, recompute PPE10 and MdAPE for each sample, and use the resulting distributions to construct 95% confidence intervals.

Table 3. Bootstrap confidence intervals (2,000 resamples)

Model	PPE10 95% CI	MdAPE 95% CI
Baseline: global median	[19.9%, 22.4%]	[31.5%, 34.7%]
Final model	[36.4%, 39.4%]	[13.0%, 13.9%]

Final model – baseline (PPE10 difference)	+16.7 percentage points [14.9%, 18.6%]	
---	--	--

The final model has PPE10 = 37.9% with a 95% confidence interval of [36.4%, 39.4%], and MdAPE = 13.4% with a 95% confidence interval of [13.0%, 13.9%]. Compared with the global-median baseline, the final model improves PPE10 by 16.7 percentage points, with a 95% confidence interval of [14.9, 18.6] percentage points. Because this interval does not include 0, the improvement over the baseline is relatively stable on the current test sample.

This bootstrap analysis mainly reflects uncertainty caused by variation in the test sample and does not represent randomness in the model training process itself. It is therefore used to judge whether the current test result is overly dependent on a small number of observations, rather than to quantify variation that would arise from retraining the model.

3.4 Error Analysis

Overall PPE10 and MdAPE describe the model's aggregate performance but do not show which property types are predicted well or where large errors occur. I therefore group the test results by price, distance from the CBD, property type, and whether the suburb was seen during training. Table 4 reports the final model's performance across these groups.

Table 4. Grouped test-set errors for the final model

Split	Group	n	PPE10 (%)	MdAPE (%)	Median bias (%)
Price band	< 0.8 M	23	26.1	14.9	+12.5
	0.8–1.2 M	824	46.2	10.7	+2.0
	1.2–2 M	1,736	41.2	12.5	–1.5
	2–4 M	1,152	31.0	15.7	–11.9
	> 4 M	172	11.0	31.9	–31.7
Distance from CBD	< 10 km	709	36.0	14.3	–1.1
	10–25 km	1,634	39.0	13.1	–2.8
	25–45 km	855	43.0	11.9	–5.2
	> 45 km	709	30.9	15.7	–11.5
Property type	Apartment / Unit / Flat	200	29.0	18.3	+6.6
	House	3,449	38.1	13.3	–5.5
	Townhouse	80	40.0	14.1	+0.7
	Other residential types	178	41.6	11.8	+0.1
Suburb seen in training?	Seen	3,759	37.8	13.4	–5.3
	Unseen	148	39.9	13.9	+6.5

Across price bands, the model performs best on mid-priced properties. For properties priced at AUD 0.8–1.2 million and AUD 1.2–2 million, PPE10 is 46.2% and 41.2% respectively, with relatively low MdAPE. In contrast, properties above AUD 4 million have PPE10 of only 11.0%, MdAPE of 31.9%, and median bias of -31.7% . This indicates that the final model performs much worse on high-end properties and tends to underestimate them.

By distance from the CBD, properties 25–45 km away perform best, with PPE10 = 43.0% and MdAPE = 11.9%. Beyond 45 km, PPE10 falls to 30.9% and median bias becomes -11.5% , indicating weaker performance for properties farther from the city centre.

Performance also differs by property type. House and Townhouse reach PPE10 values of 38.1% and 40.0%, while Apartment / Unit / Flat reaches 29.0%. Apartments have a median bias of $+6.6\%$, indicating some overall overestimation, while Houses have a median bias of -5.5% , indicating some underestimation.

I also compare suburbs that were and were not seen in the training set. PPE10 is 37.8% for seen suburbs and 39.9% for unseen suburbs, with no obvious performance gap. This suggests that, even without using suburb name directly, the final model can make predictions for unseen suburbs using latitude, longitude, distance to the CBD, income, and population. However, there are only 148 unseen-suburb records in the test set, so this result should be interpreted cautiously.

Finally, I inspect the transactions with the largest relative errors. The largest errors mainly occur in expensive Houses with many bedrooms, including several 7–10 bedroom properties. These cases are rare in the dataset and their prices may depend on factors not captured by the available features, making them difficult for the model to estimate accurately.

LightGBM gain importance assigns the largest share to `km_from_cbd` at 33.2%, followed by number of bathrooms (12.5%), property size (11.5%), and longitude (10.8%). This shows that the model makes substantial use of location and property attributes. However, feature importance only indicates how much the model uses each variable; it does not establish a causal relationship with house prices.

3.5 Quantile Calibration and a Lending Perspective

In addition to a point estimate, the project uses quantile regression to represent prediction uncertainty. To check whether the quantiles are calibrated, I compare each predicted quantile with the proportion of actual test sale prices below it. For example, if q_{10} is well calibrated, about 10% of true sale prices should fall below q_{10} ; similarly, q_{90} should be close to 90%. Table 5 shows the observed results.

Table 5. Calibration of the quantile models

τ	Proportion of test sale prices below this quantile estimate	Pinball loss (log)
0.10	0.109	0.0365
0.25	0.220	0.0660
0.50	0.379	0.0920
0.75	0.582	0.0813
0.90	0.809	0.0513

For q10, the observed proportion is 10.9%, which is very close to the theoretical 10%. However, at higher quantiles the observed proportion increasingly falls below the target. For example, q50 should have about 50% of sale prices below it, but the observed value is only 37.9%; q90 is 80.9%, also below the theoretical 90%. This means that predictions at the higher quantiles are systematically too low.

The q10–q90 interval should theoretically provide 80% coverage, but it covers only about 70% of the test transactions. The interval is therefore not fully calibrated, and more true sale prices fall outside it than expected. One possible reason is the strong market shift between training and test periods, which still causes some underestimation by the quantile models at the higher 2021 market level.

To examine the trade-off between different valuation positions, I compare the point estimate, q50, q25, and q10. The results are shown in Table 6.

Table 6. Lending perspective: overestimation and underestimation at different valuation points

Valuation used	Overestimation rate (%)	Overestimation > 10% (%)	Underestimation > 10% (%)	PPE10 (%)
Point estimate (L2)	40.7	23.6	38.5	37.9
q50	37.9	21.6	40.6	37.8
q25	22.0	9.7	57.7	32.6
q10	10.9	4.1	74.4	21.5

The point estimate has PPE10 = 37.9%, with 23.6% of properties overestimated by more than 10% and 38.5% underestimated by more than 10%. Using q25 reduces overestimation greater than 10% to 9.7%, but increases underestimation greater than 10% to 57.7%, and PPE10 falls to 32.6%. With q10, only 4.1% of properties are overestimated by more than 10%, but 74.4% are underestimated by more than 10%.

These results show that lower quantiles produce more conservative valuations, but at the cost of more underestimation and lower overall prediction accuracy. No single quantile is best for every use case; each reflects a different valuation preference. In the deployment function, the point estimate is the main prediction, q10–q90 is used as the prediction range, and q25 is provided as a relatively conservative reference price.

Figure 6 plots predictions after sorting test transactions by the true price. Most actual sale prices are near the predicted range, but for some high-value properties the actual sale price is clearly above the prediction interval. This is consistent with the earlier error analysis and shows that the model still has substantial uncertainty for expensive properties.

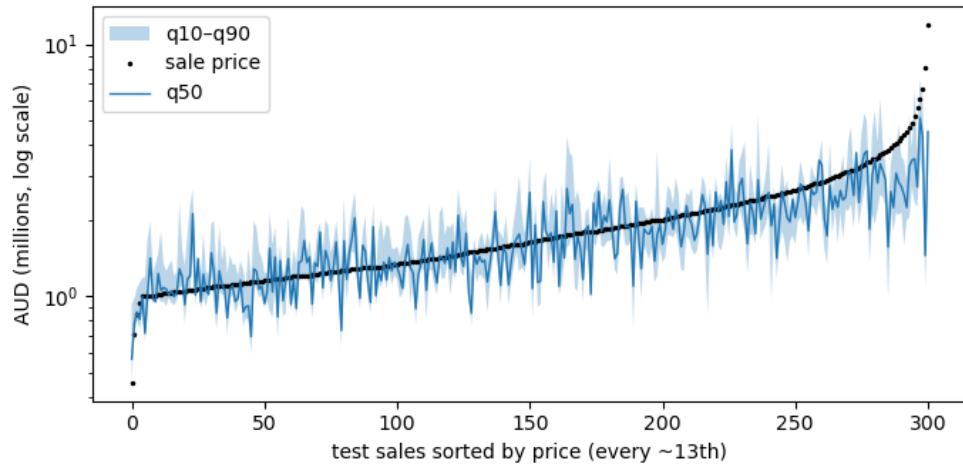


Figure 6. Test-set house-price predictions. The shaded region is the $q10$ – $q90$ prediction interval, the blue line is $q50$, and black points are actual sale prices. For readability, approximately one record out of every 13 is shown.

4 Discussion

4.1 Answer to the Research Question

The main research question is whether a model trained on 2016–2020 historical transactions can accurately predict 2021 house prices, and how prediction error caused by an overall market shift can be reduced.

The results show that if a model must learn the market price level only from historical data, a major market change can create large systematic errors. For example, the original LightGBM achieves only 22.5% PPE10 on the test set and has a median bias of -18.2% , indicating substantial underestimation in the 2021 market. Ridge and MLP perform better, but they still do not fully solve the market-shift problem.

Separating overall market movement from property-specific value leads to a clear improvement. After adjusting the target with the property price index, the final LightGBM reaches 37.9% PPE10, 13.4% MdAPE, and -4.7% median bias. Ridge reaches 37.3% PPE10 with the same target adjustment. This shows that, in this project, how the training-to-test market shift is handled is at least as important as which model family is chosen.

In the more realistic one-quarter-lag experiment, PPE10 is 35.7%, MdAPE is 14.5%, and median bias is -7.9% . Performance is lower than with the contemporaneous index but still clearly better than the original LightGBM. This shows that index adjustment remains useful even after accounting for publication delay.

4.2 Implications

First, systematic overestimation or underestimation matters when the market changes. In this project, the original LightGBM substantially underestimates the 2021 test market, while median bias is much smaller after index adjustment. In practice, it is therefore important to monitor not only accuracy metrics such as PPE10 and MdAPE but also whether predictions become systematically biased. If the market environment changes substantially, the model may need retraining or updated market information.

Second, the training loss is not the same as the practical prediction objective. Different loss functions produce fairly similar PPE10 values, and a lower training loss does not necessarily produce a higher PPE10. I therefore use the training loss for optimisation but use PPE10, MdAPE, and median bias for model selection and final evaluation.

Third, the data-splitting strategy has a large effect on the reported result. The random split reaches 42.3% PPE10, compared with 37.9% for the final model under the chronological split. However, the random split places some future-period transactions into the training set and makes the training and test distributions more similar. For time-varying data such as house prices, chronological train/validation/test splitting is therefore more representative of real future-market prediction.

A further important finding is that a more complex model does not automatically produce a better result. I initially expected LightGBM to clearly outperform Ridge, but the original LightGBM performs worse in the 2021 test period. After index adjustment, LightGBM and Ridge reach very similar PPE10 values of 37.7% and 37.3%. The main improvement in this project therefore comes from redesigning the prediction target rather than simply replacing the model with a more complex one.

4.3 Limitations

This project still has several limitations:

1. A single overall property price index is used. The same market index is applied to all properties, but the error analysis shows different behaviour for high-value properties, properties far from the CBD, and different property types. This suggests that market trends may differ by region and property segment. Future work could use more granular regional or property-type indices.
2. Location is represented only at suburb level. The latitude and longitude values are suburb centroids, so all properties within a suburb share the same coordinates. Real house prices are also affected by specific streets, school zones, transport access, views, and other factors that are not present in the current data.
3. The model is mainly tested in a rising market. The dataset covers 2016–2021, and the test period falls within the strong 2021 upswing. The current results therefore show performance under this type of market change, but they do not establish that the index-adjustment method would perform equally well in a falling or long-term stable market.
4. The property price index itself still needs further verification. `property_inflation_index` is an important input to the final model, but the dataset does not provide a complete source, publication schedule, or property-type coverage for the index. The one-quarter-lag experiment considers the possible effect of publication delay, but real deployment would still require confirmation of the index source, definition, and actual availability date.
5. Data quality is still limited for high-end and unusual properties. The cleaning process removes clearly implausible records, but some expensive properties may depend on special factors that are not captured by ordinary property features. The error analysis also shows weak performance for properties above AUD 4 million and some Houses with 7–10 bedrooms. The current model is therefore more suitable for ordinary residential properties, while high-end or unusual properties should be treated with greater caution.

6. Repeat sales of the same property cannot be identified. The current dataset has no unique property ID, so it is not possible to determine whether the same property appears in multiple years. If the same property appears in both training and test sets, evaluation may be affected. With full addresses or unique property identifiers, future work could detect and control for repeat sales.

4.4 Future Improvements

1. Use more granular property price indices. The current model uses one overall index to represent market change, but different regions, property types, and price bands may move differently. Future work could build separate indices for Houses, Apartments, price segments, or geographic regions and update them regularly using recent transactions.
2. Improve quantile interval calibration. The current q10–q90 interval theoretically represents 80% coverage, but actual test coverage is only about 70%. Future work could calibrate the quantile models so that the interval better matches actual transaction outcomes and reduce inconsistent behaviour across quantiles.
3. Improve handling of high-value and unusual properties. The error analysis shows weaker performance on expensive properties and some homes with many bedrooms. Future work could add stricter data checks and features that better describe high-end housing, such as more detailed location, property quality, or other property characteristics.
4. Retest the model using newer data. The current model is mainly tested during the rising 2021 market. Once data from 2022 and later are available, the same chronological training and testing process could be repeated to evaluate whether index adjustment remains effective in falling or relatively stable markets.

5 Implementation Log

This section is based on the project's journal/implementation_log.md and records, in chronological order, the main steps from data exploration and model building to the final improvements, including problems encountered, methods attempted, and the reasons for each change. It also documents how AI tools were used during the project.

ChatGPT was used to assist with parts of the coding, debugging, and code organisation. However, the data-cleaning rules, model design, experimental plan, evaluation metrics, and final interpretation of results were decided by me based on the experimental evidence. For AI-assisted code, I verified correctness by running the code, comparing results with library implementations, and checking consistency across experiments. The implementation log also records unsuccessful approaches and subsequent changes to reflect the actual development process.

5.1 Development Process and Challenges

Stage	What I did	Challenge and response
Topic selection and data exploration	Selected Option 2, defined the task as Sydney residential price prediction, and inspected the raw dataset of 11,160 rows and 17 columns.	The data are not a completely clean “toy dataset”. Exploration showed a strongly right-skewed price distribution with values up to AUD 60 M, which motivated using $\log(\text{price})$ as the main target.

Publishing to Colab	Created a public GitHub repository and Colab notebook, with data loaded directly from the repository.	The first git push used the wrong GitHub account. I reconfigured the personal access token and confirmed that Colab could run successfully.
Task definition and cleaning	Defined the model inputs and outputs, removed non-residential property types and clearly implausible records, reducing the data from 11,160 to 10,864 rows.	Cleaning required distinguishing genuine expensive properties from data errors. For example, the Woollahra high-value sale was extreme but plausible and was retained, while clearly impossible records such as 47 bedrooms and 46 bathrooms were removed.
Chronological split	Split the data by transaction date into training, validation, and test sets.	I initially considered a random split but found that it placed many 2021 records in training, allowing the model to see the future market. I therefore used chronological splitting and kept the random split only as a control experiment.
Rubric check, Sections 0–3	Checked the task definition, model understanding, and evaluation method against the Assignment rubric.	The check showed that reporting accuracy alone was not enough, so I added training/deployment I/O, model principles, ablation experiments, and error analysis.
Metrics and baselines	Implemented PPE10, MdAPE, median bias, and simple baselines.	The suburb-median baseline did not outperform the global-median baseline, which suggested that the main issue might be the market shift between training and test periods rather than location alone.
Ridge	Completed feature encoding and implemented Ridge using the closed-form solution, sklearn, and manual optimisation.	Feature-name and encoding issues occurred during implementation. After adjusting one-hot encoding, the NumPy closed-form solution matched sklearn almost exactly, validating the implementation.
Manual GD / Adam	Implemented Gradient Descent and Adam on the same Ridge loss.	Standard GD converged slowly, while Adam approached the closed-form solution after about 1,500 iterations. This experiment helped me understand differences in optimiser convergence.
LightGBM	Trained LightGBM on the same data and tuned num_leaves and early stopping.	LightGBM performed well on 2020 data but substantially underestimated the 2021 test set. Additional experiments showed that the main issue was the shift in market level rather than the model itself.
MLP	Built a 616→128→64→1 neural network in PyTorch and repeated training with five random seeds.	Training error kept decreasing but validation error rose early, so I used early stopping. Results also varied across random seeds.
Ablation and final model	Used a consistent ablation framework to compare the prediction target, market variables, suburb, and data-splitting strategy.	The most important finding was that the index-adjusted target increased LightGBM PPE10 from 22.5% to 37.7%, and Ridge also improved to 37.3%. This shows that most of the improvement came from target design rather than simply changing models.
Loss vs objective, quantiles, and error analysis	Compared different loss functions, trained quantile models, and analysed prediction errors across property groups.	The lowest training loss did not necessarily produce the highest PPE10, and q10–q90 coverage was below its theoretical level, showing that the prediction intervals still require further calibration.
Deployment function	Integrated the final pipeline into estimate_price() and built a simple interactive valuation tool.	Deployment requires validating user inputs and automatically adding suburb-level information. For years after 2022, when no observed index is available, the tool provides scenario analysis only, not genuine future-price forecasts.
Interactive	Built a simple ipywidgets interface for	Observed property-price indices are used for 2016–2021. For

valuation tool	viewing estimated values from 2016 to 2030.	2022 onward, there are no observed index values, so scenario indices are generated from an assumed growth rate. The interface clearly labels these as hypothetical scenarios rather than real forecasts.
----------------	---	--

5.2 Use of AI Tools

I used ChatGPT as an implementation tool during development. I made the decisions about the project topic, dataset selection, problem definition, cleaning rules, split strategy, evaluation metrics, and which experiments to run. AI was used to turn those decisions into code more quickly, fix errors, and draft explanatory text. I ran and checked every code section, and all numerical results in the report come from my own notebook runs.

Area	How I checked / changed it	AI output
Topic and data	Selected Option 2 and the Sydney house-price task, confirmed that the dataset is real market data rather than a toy dataset, and decided to train on $\log(\text{price})$	Assisted with code for checking the dataset overview
Task definition and cleaning	Defined training- and deployment-stage inputs and outputs; decided which types are outside the “single residential property” scope; set limits for bedrooms, bathrooms, parking, and area; decided to retain the genuine AUD 17 M sale	Implemented <code>clean()</code> and the input-specification table according to my rules
Data split and metrics	Decided to use chronological rather than random splitting, required a leakage comparison, and selected PPE10 and MdAPE as primary metrics	Implemented the split and metric functions; I verified <code>metrics()</code> using hand-constructed examples
Models	Decided to compare Ridge, LightGBM, and MLP, and required manual Gradient Descent and Adam in addition to the closed-form Ridge solution to demonstrate the training process	Drafted model and training-loop code; I checked agreement with sklearn (7e-15) and confirmed that early stopping kept the best weights
Diagnosis and improvement	After LightGBM lost to Ridge, I decided to run two diagnostics (macro-value replacement and within-range re-testing) to confirm an extrapolation problem rather than a bug; I then decided to use an index-adjusted target and test a lagged index	Implemented the ablation framework; I read the ablation table line by line and rechecked unusual results

Evaluation and deployment	Decided to include loss-function comparisons, quantile intervals, error analysis by price/distance/type, bootstrap intervals, a callable estimate_price() function for single properties, and an interactive interface; future years had to be labelled as scenarios	Implemented these functions; I checked the calibration table and input validation
Documentation	The structure of the log and report, discussion, limitations, and knowledge gaps are in my own words	Drafted some explanatory text; I rewrote it and then used AI to help correct and tidy the final report formatting.

5.3 Knowledge Gaps

The table below lists components that I used, can explain why they are needed, and verified to work as intended, but do not yet fully understand.

Component	Why it is needed	How I checked it	What I still do not fully understand
<code>pd.to_datetime(format="%d/%m/%y")</code>	Correct date parsing is the basis of the chronological split and the year/month features	Checked the parsed date range and sort order	Need a deeper understanding of how different date formats and abnormal inputs are handled
property_inflation_index	The final model relies on it to represent overall market price change	Checked how the index changes over time and tested a one-quarter lag	Still need to confirm the original source, publication timing, and exact coverage of the index
Property-size cleaning range	Directly affects which transaction records are retained	Inspected the removed outliers and confirmed that clearly implausible areas were excluded	The current 20–10,000 m ² range is mainly based on the observed data and still lacks an external standard
PPE10 10% threshold	One of the main evaluation metrics in this project	Compared PPE10 with MdAPE and other error metrics	±10% is a practical evaluation band, but more industry evidence would be needed to justify this threshold
Adam parameter updates	Used in the Ridge experiment and MLP training	Implemented Adam manually and compared it with the Ridge closed-form solution to confirm convergence	Need a deeper understanding of Adam convergence under different learning rates and hyperparameters
LightGBM categorical feature handling	Affects how the model uses categorical variables such as type	Confirmed experimentally that the model trains normally and gives stable outputs	Need a deeper understanding of how LightGBM internally finds the best categorical splits
MLP randomness	Different initialisations can produce different predictions	Repeated training with five random seeds and compared the results	More repetitions would be needed to assess stability more thoroughly

Pinball loss and quantile prediction	Used for the q_{10} – q_{90} prediction interval	Checked the observed coverage of different quantiles on the test set	Need a deeper understanding of interval calibration and how to address observed coverage below the nominal level
--------------------------------------	--	--	--

References

- alexlau203. (n.d.). *Sydney house prices* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/alexlau203/sydney-house-prices>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. <https://doi.org/10.48550/arXiv.1412.6980>
- Koenker, R., & Bassett, G., Jr. (1978). Regression quantiles. *Econometrica*, 46(1), 33–50.
<https://doi.org/10.2307/1913643>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Zillow. (2011). Upgrading the Zestimate. *Zillow Research*.